

Assignment 7

Secure Network Application Development Using TCP

Jitupan Mondal

Roll No: EEB24023

B.Tech Electrical Engineering, Tezpur University

ISEA Phase 3 Cybersecurity Internship

System Details

- **OS:** Linux jitupan-mondal-QEMU-Virtual-Machine 7.0.0-22-generic (Ubuntu, aarch64)
- **Python Version:** 3.14.4
- **Network Emulator:** Mininet
- **Packet Analyzer:** Wireshark
- **GUI Toolkit:** Tkinter
- **Hashing Algorithm:** SHA-256

Contents

1	Objective	3
2	Security Features Implemented	3
3	System Architecture	3
3.1	Server-Side Architecture	4
3.2	Client-Side Architecture	4
4	Implementation	4
4.1	Credential Storage	4
4.2	Authentication Workflow	5
4.3	JSON Login Protocol	5
4.4	Duplicate Login Prevention	5
4.5	Security Logging	6
5	Testing	6
6	Wireshark Verification	9
7	Conclusion	11

1. Objective

The objective of this assignment is to enhance the GUI-based multi-client TCP chat application developed in Assignment 6 by implementing practical application security mechanisms, with an emphasis on authentication, secure credential storage, and session integrity rather than advanced cryptography. Specifically, the assignment requires username and password based authentication, SHA-256 password hashing, duplicate login prevention, and security event logging, all layered on top of the existing GUI and networking implementation without altering its core architecture.

2. Security Features Implemented

The following application-layer security mechanisms were added to the Assignment 6 implementation:

- **Username/password authentication** – every client must supply valid credentials before being admitted to the chat session.
- **SHA-256 password hashing** – passwords are never stored or compared in plaintext; the server hashes incoming passwords with `hashlib.sha256()` and compares the resulting digest against a precomputed hash stored in `users.json`.
- **JSON-based login protocol** – the client and server exchange a structured JSON login request and response, replacing the raw username string used in Assignment 6.
- **Server-side authentication enforcement** – all credential verification is performed exclusively on the server, since the client cannot be trusted to enforce access control on its own.
- **Duplicate login prevention** – the server tracks currently authenticated usernames in memory and rejects a second simultaneous login attempt for a username that is already connected.
- **Security event logging** – authentication outcomes (`LOGIN_SUCCESS`, `LOGIN_FAILED`, `DUPLICATE_LOGIN`, `LOGOUT`) are recorded to a dedicated `security_log.txt` file, kept separate from the connection log (`server_log.txt`) inherited from Assignment 6.
- **Wireshark-based verification** – the authentication workflow was validated at the packet level to confirm correct protocol behaviour under successful, failed, and duplicate login conditions.

3. System Architecture

The application continues to follow the client-server architecture over TCP sockets established in Assignment 6, tested using the Mininet topology `sudo mn --topo single,5` summarised in Table 1. The server listens on port 5000 and continues to spawn one dedicated handler thread per connecting client.

Table 1: Mininet topology used for testing

Host	Role	IP Address	Username
h1	Chat Server	10.0.0.1	–
h2	Client A	10.0.0.2	Amit
h3	Client B	10.0.0.3	Priya
h4	Client C	10.0.0.4	Rahul
h5	Client D	10.0.0.5	Sneha

3.1 Server-Side Architecture

The server (h1) retains its Assignment 6 responsibilities of accepting multiple TCP connections and maintaining a shared `clients` dictionary protected by a lock. In addition, it now performs the following on every new connection:

1. Accept the TCP connection and spawn a handler thread.
2. Parse the client's JSON login request.
3. Hash the supplied password using SHA-256 and compare it against the stored hash in `users.json`.
4. Reject the connection if credentials are invalid or if the username is already logged in.
5. Admit the client into the chat session only after successful authentication.

3.2 Client-Side Architecture

The client retains the two-layer design established in Assignment 6:

1. **Networking layer** (`NetworkClient` class) – now performs a JSON-based login handshake in addition to its existing responsibilities of message sending and background message reception.
2. **Presentation layer** (`LoginWindow` and `ChatWindow` classes) – the `LoginWindow` was extended with a password field, while the `ChatWindow` remains functionally unchanged from Assignment 6.

As in Assignment 6, the two layers communicate exclusively through a thread-safe `queue.Queue`, polled by the GUI thread via `root.after()`, ensuring that Tkinter widgets are never modified from outside the main thread.

4. Implementation

4.1 Credential Storage

User credentials are stored in `users.json` as a mapping of username to SHA-256 password hash. Plaintext passwords are never written to disk at any point in the application.

```
{
  "amit": "1fec7401b302e76333629c9f68415b7718769e072f0613e54726143cda06d789",
  "priya": "0a8b8dfad3f637d7d30fed7b108c5c5986c4775d14cab26ec9279866eba99116",
  "rahul": "687f6da20de59091e67f594827cdee268125feb3aed1c3bad77d0f6761eb198a",
  "sneha": "59b6bc9484a2c8ea7c20a3c0778088295e6f8e62a6a90b6d535b3b6bceb2fcd0"
}
```

Listing 1: Structure of `users.json`

4.2 Authentication Workflow

Authentication proceeds as follows: the client connects to the server; the client sends a JSON login request; the server parses the JSON payload; the server hashes the supplied password; the server compares the hash against `users.json`; the server checks for an existing session under the same username; and finally the server either admits the client into the chat session or returns a JSON error response.

```
def hash_password(password):
    return hashlib.sha256(password.encode("utf-8")).hexdigest()

def verify_credentials(username, password):
    if username not in USERS:
        return False
    return USERS[username] == hash_password(password)
```

Listing 2: SHA-256 password verification on the server

4.3 JSON Login Protocol

The client and server exchange newline-delimited JSON messages during login. Table 2 summarises the payloads observed for each outcome.

Table 2: JSON login protocol payloads

Scenario	Payload
Login request (client → server)	{ "action": "login", "username": "amit", "password": "amit123" }
Successful login (server → client)	{ "status": "success", "message": "Login successful" }
Invalid credentials (server → client)	{ "status": "error", "reason": "invalid_credentials" }
Duplicate login (server → client)	{ "status": "error", "reason": "duplicate_login" }

4.4 Duplicate Login Prevention

The server maintains an in-memory set of currently logged-in usernames. Before admitting a new session, it checks whether the requested username already exists in this set; if so, the connection is rejected with a `duplicate_login` error rather than being allowed to silently override the existing session. The username is removed from the set only when

the corresponding client disconnects, ensuring that a user can log in again immediately after a genuine logout.

4.5 Security Logging

All authentication-related events are written to `security_log.txt`, separate from the connection log inherited from Assignment 6. Logged events include `LOGIN_SUCCESS`, `LOGIN_FAILED`, `DUPLICATE_LOGIN`, and `LOGOUT`, each recorded with a timestamp, username, and source IP address. Passwords are never written to any log file, whether in plaintext or hashed form, in accordance with the assignment's security logging requirement.

5. Testing

The application was tested end-to-end in Mininet using the topology and usernames from Table 1. Three authentication scenarios were exercised and captured as screenshots: successful authentication, authentication failure due to invalid credentials, and rejection of a duplicate login attempt.

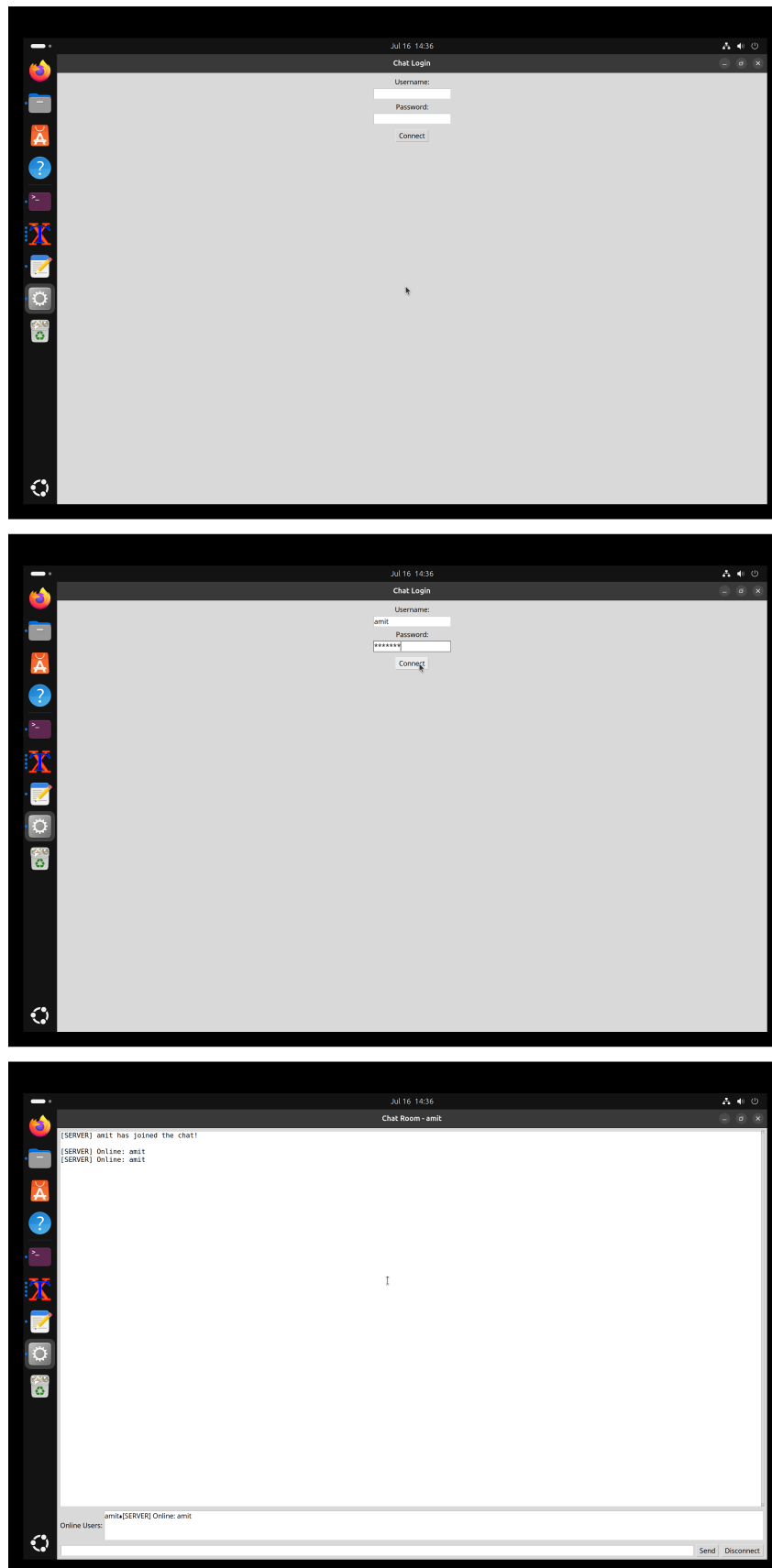


Figure 1: Successful user authentication

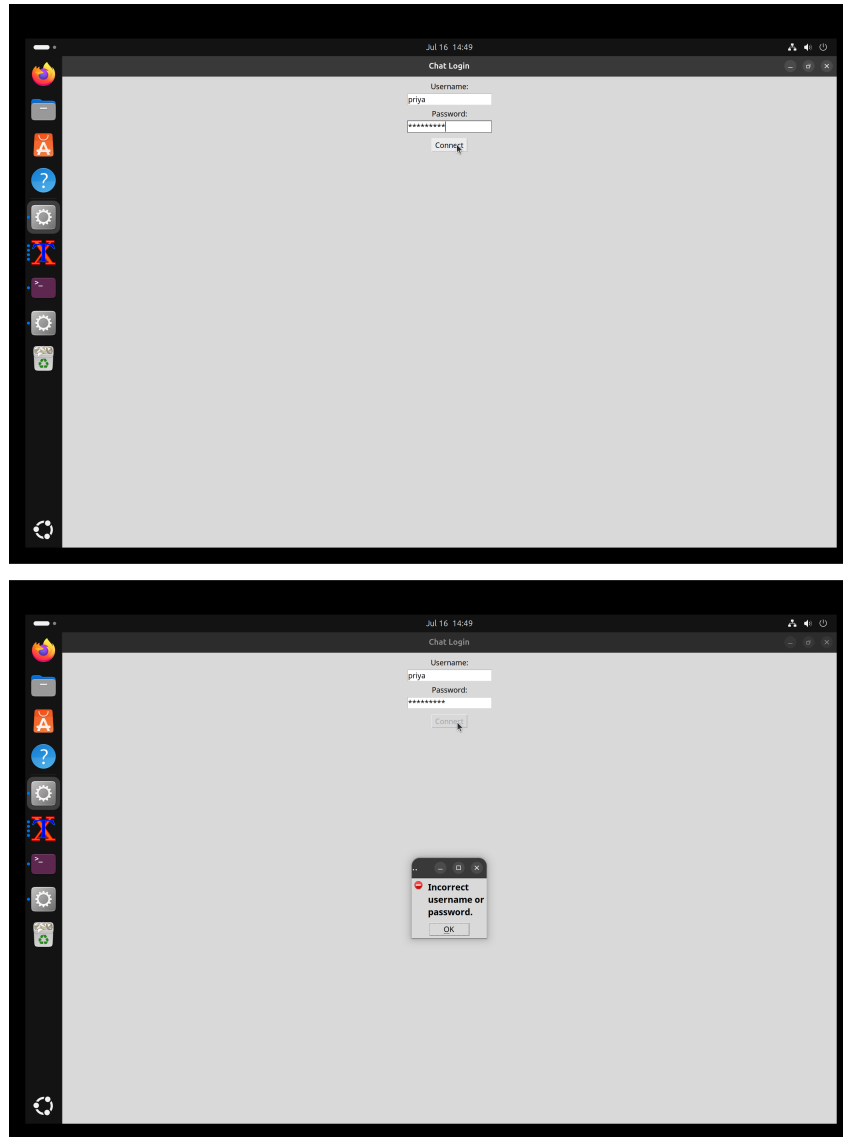


Figure 2: Authentication failure due to invalid credentials

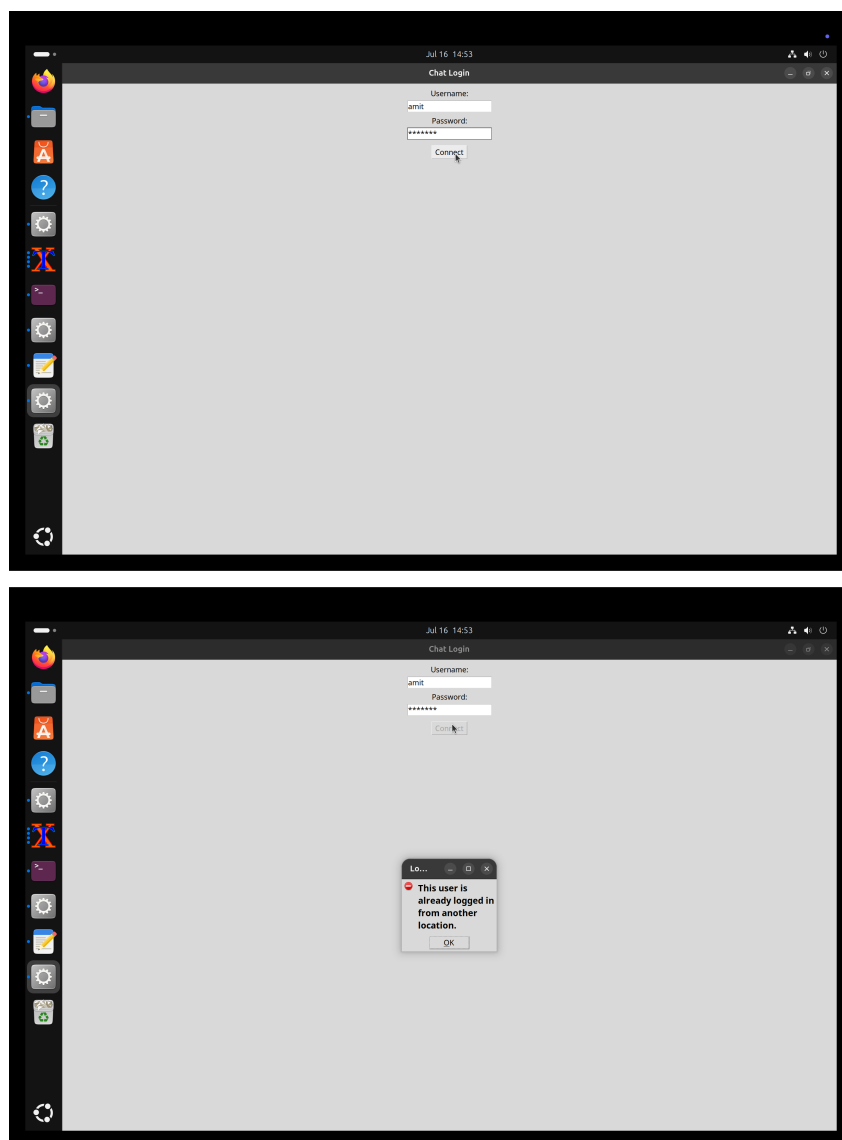


Figure 3: Duplicate login prevention mechanism

The security log independently confirms correct recording of all three authentication outcomes:

```
HH:MM:SS,LOGIN_SUCCESS,amit,10.0.0.2,authenticated
HH:MM:SS,LOGIN_FAILED,priya,10.0.0.3,invalid credentials
HH:MM:SS,DUPLICATE_LOGIN,amit,10.0.0.4,rejected - already online
```

Listing 3: Excerpt from `security_log.txt`

The connection log (`server_log.txt`) inherited from Assignment 6 continued to record **CONNECTED** and **DISCONNECTED** events correctly, confirming that the authentication layer was added without disrupting existing connection-tracking functionality.

6. Wireshark Verification

Network traffic was captured on interface `h1-eth0` using `tcpdump` with the filter `tcp port 5000`, then analysed offline in Wireshark using the equivalent display filter `tcp.port ==`

5000. Four key aspects of the authentication workflow were isolated and screenshotted.

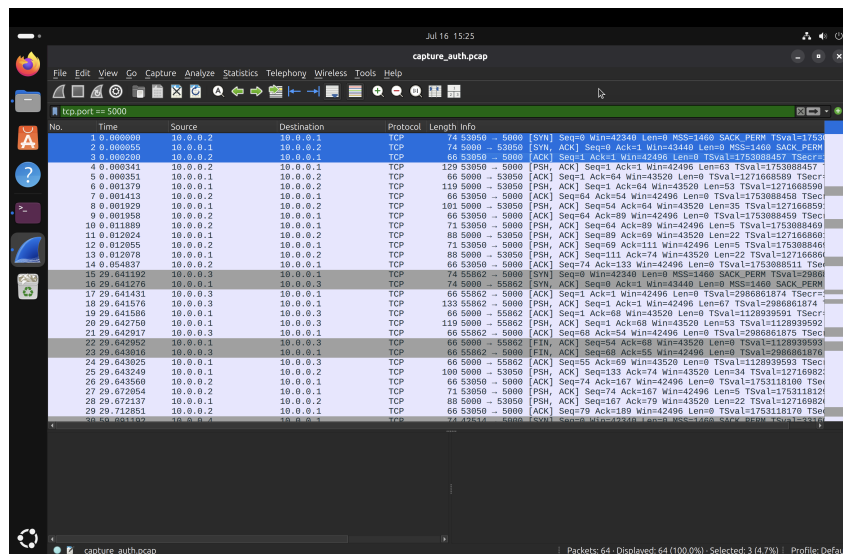


Figure 4: TCP three-way handshake during client connection

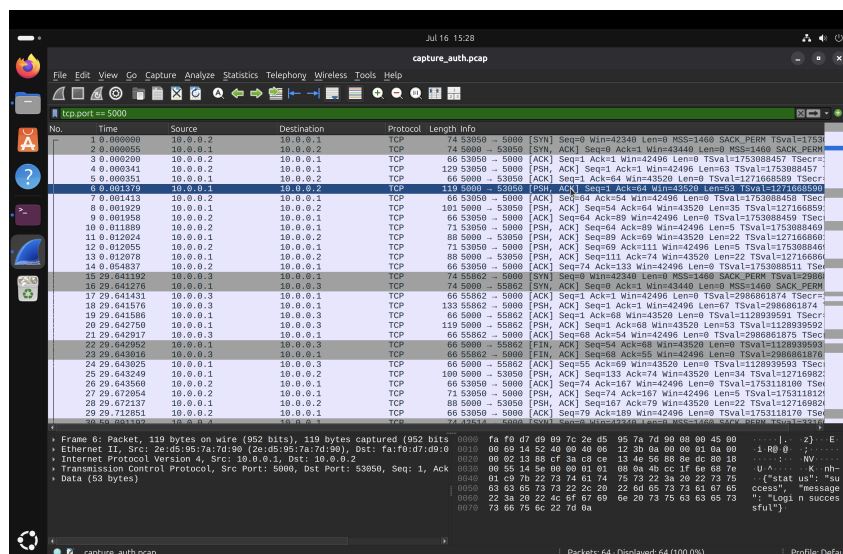


Figure 5: Wireshark capture of successful login authentication

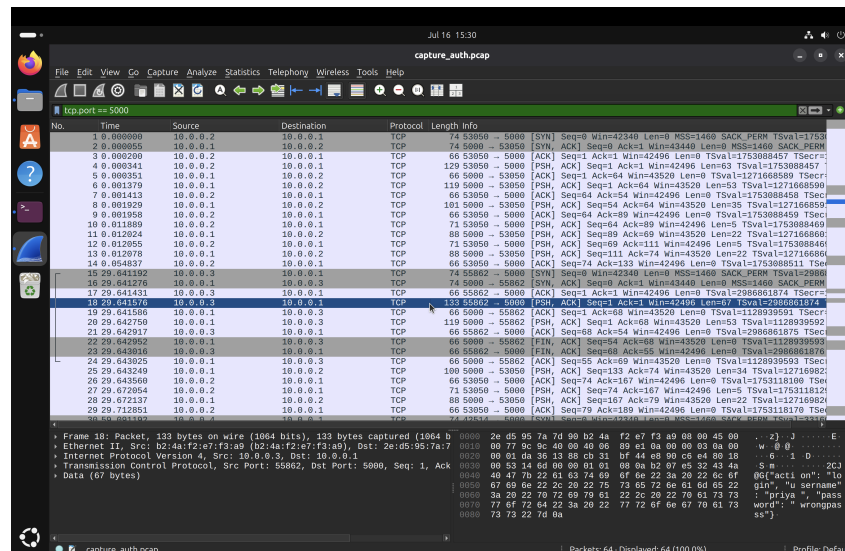


Figure 6: Wireshark capture of failed login authentication

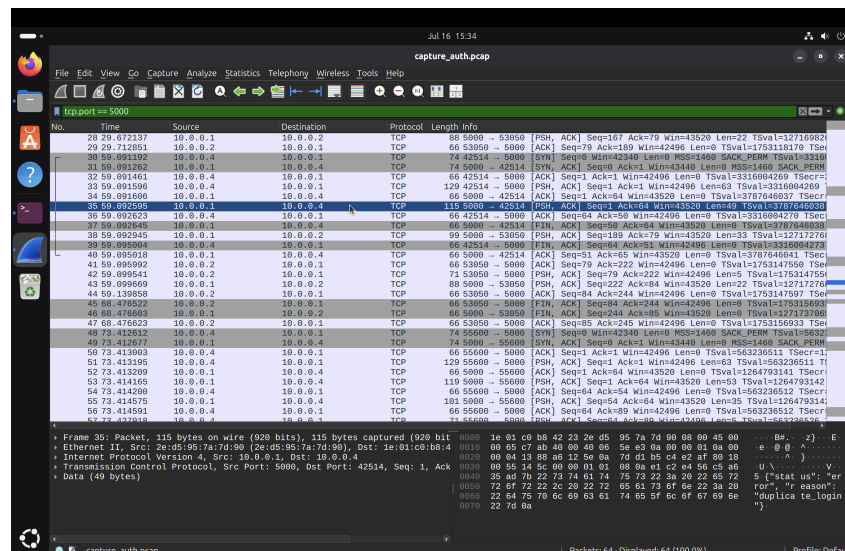


Figure 7: Wireshark capture of duplicate login rejection

Following the TCP stream for each login attempt confirms that the client transmits the JSON login request – including the plaintext password – immediately after the three-way handshake completes, and that the server responds with the corresponding JSON status message. The captures show the login request and response payloads listed in Table 2 appearing on the wire exactly as implemented. As noted in Section 7, these packets confirm that while passwords are hashed before storage, they are transmitted in plaintext over the socket, since Transport Layer Security (TLS) was intentionally excluded from the scope of this assignment.

7. Conclusion

This assignment successfully extends the Assignment 6 GUI-based multi-client chat application with practical application-layer security mechanisms, including username/password

authentication, SHA-256 password hashing, a structured JSON login protocol, server-side duplicate login prevention, and dedicated security event logging. All enhancements were implemented on top of the existing architecture without modifying the underlying GUI layout or the thread-safe queue-based communication model established in Assignment 6. End-to-end testing across a five-host Mininet topology, corroborated by Wireshark packet-level verification, confirmed correct behaviour for successful authentication, invalid credential rejection, and duplicate login prevention. Passwords are stored exclusively as SHA-256 hashes and are never logged in plaintext; however, since TLS was outside the scope of this assignment, credentials remain visible in plaintext at the network layer, representing a clear direction for future security hardening of the application.